# SIMATS ENGINEERING

## SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

**Department of Computer Science and Engineering**

---

# ASSESSMENT TOOL 2 – SCHEMA ANALYSIS AND VIVA VOCE

| | | | |
|---|---|---|---|
| **Course Code:** | DSA05 | **Course Title:** | Query Processing for Data Science With Real Time Applications |
| **CO Assessed:** | CO2 | **Assessment:** | Tool 2 – Schema Analysis and Viva Voce |
| **Weightage:** | | **Total Marks:** | 35 Marks |
| **CO2 Statement:** Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6) | | | |
| **Student Name:** | Sania Herbert | **Reg. No.:** | 192324062 |
| **Section / Batch:** | 2023/AI&DS | **Date of Submission:** | 8.6.26 |
| **Faculty In-charge:** | **Dr. B.SHYAMALA GOWRI** | **Project Mode:** | Individual Submission |

**Code of Conduct**

I Sania Herbert(192324062) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Sania

| Identification of Entities and Attributes (5 Marks) | Understanding of Relationships (5 Marks) | Analysis of Keys and Constraints (5 Marks) | Detection of Design Issues (10 Marks) | Viva Voce (5 Marks) | Total (35 Marks) |
|---|---|---|---|---|---|
| | | | | | |

**1. Hospital Management System**
A hospital database contains:

- Patient(PatientID, Name, Age)
- Doctor(DoctorID, DoctorName, Specialization)
- Appointment(AppointmentID, PatientID, DoctorID, Date)
- Prescription(PrescriptionID, AppointmentID, Medicine)

Multiple patients can consult the same doctor.

**Viva Questions**
Q1. What problems might occur if prescription details are stored directly in the Appointment table?
Q2. How would the schema change if a prescription contains multiple medicines?
**SOLUTION:**
**Part A: Identification of Entities and Attributes**
**1. Patient**

| **Attribute** | **Description** |
|---|---|
| PatientID (PK) | Unique ID of patient |
| Name | Patient Name |
| Age | Patient Age |

**2. Doctor**

| **Attribute** | **Description** |
|---|---|
| DoctorID (PK) | Unique ID of doctor |
| DoctorName | Doctor Name |
| Specialization | Medical specialization |

**3. Appointment**

| **Attribute** | **Description** |
|---|---|
| AppointmentID (PK) | Unique Appointment ID |
| PatientID (FK) | References Patient |
| DoctorID (FK) | References Doctor |
| Date | Appointment Date |

**4. Prescription**

| **Attribute** | **Description** |
|---|---|
| PrescriptionID (PK) | Unique Prescription ID |
| AppointmentID (FK) | References Appointment |
| Medicine | Prescribed Medicine |

**Part B: Relationships**
**Relationship 1**
Patient → Appointment
One Patient can have many Appointments.
Relationship:

Patient (1) ------ (M) Appointment

**Relationship 2**

Doctor → Appointment

One Doctor attends many Appointments.

Relationship:

Doctor (1) ------ (M) Appointment

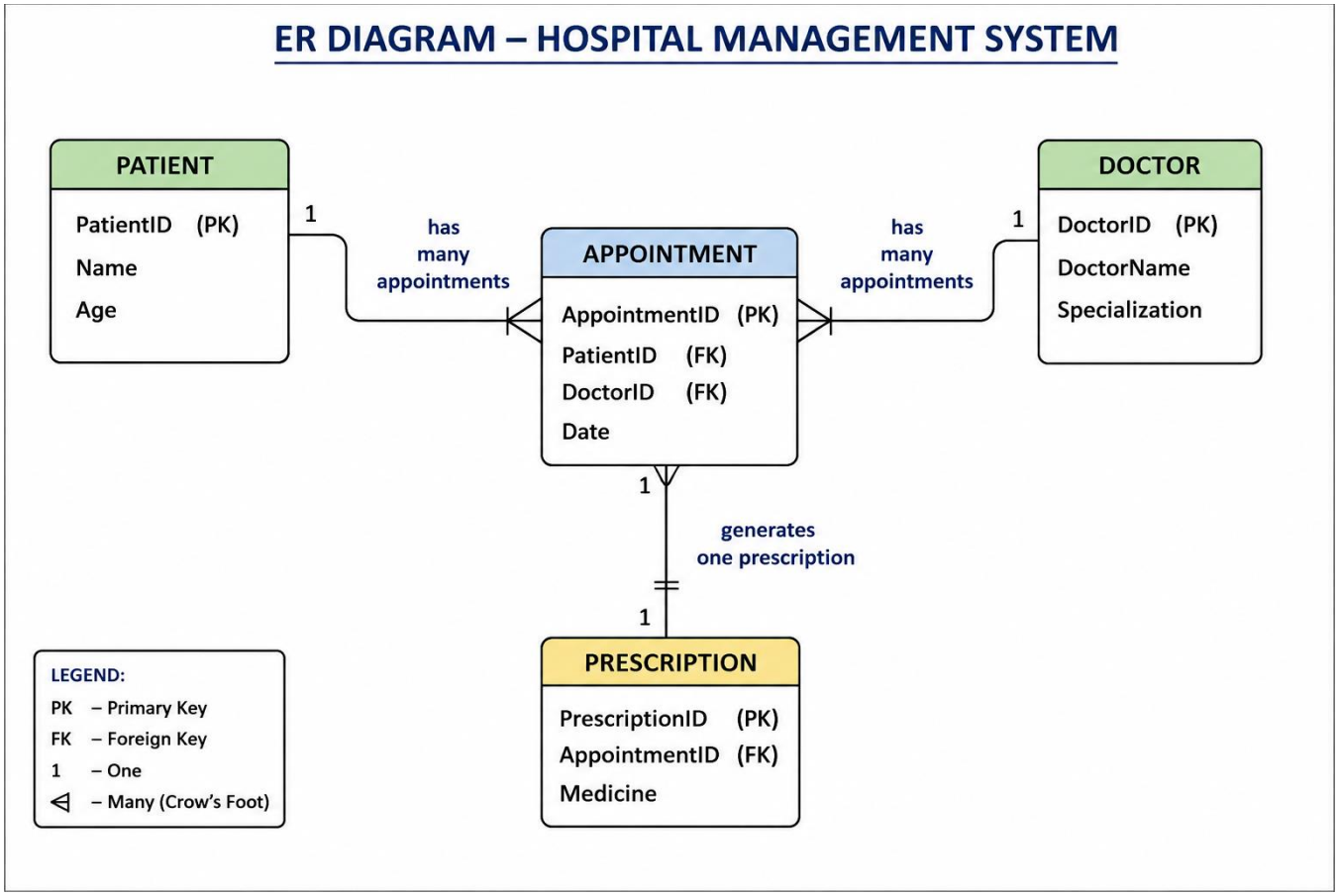
**Relationship 3**

Appointment → Prescription

One Appointment generates one Prescription.

Relationship:

Appointment (1) ------ (1) Prescription




**Part C: Keys and Constraints**

Primary Keys

- PatientID
- DoctorID
- AppointmentID
- PrescriptionID

---

**Foreign Keys**

Appointment.PatientID → Patient.PatientID

Appointment.DoctorID → Doctor.DoctorID

Prescription.AppointmentID → Appointment.AppointmentID

---

**Constraints**

- PatientID cannot be NULL
- DoctorID cannot be NULL
- Appointment Date should not be NULL
- Medicine should not be NULL
- Foreign key integrity must be maintained

**Part D: Design Issues**

**Issue 1**

If Patient information is stored repeatedly in Appointment,
→ Data Redundancy occurs.

**Issue 2**

If Doctor specialization is repeated in every Appointment,
→ Update Anomaly occurs.

**Issue 3**

If Prescription is stored inside Appointment,
→ Difficult to store multiple medicines.

**Issue 4**

Deleting an appointment may accidentally remove prescription history.
Solution
Separate the database into four normalized tables:

- Patient
- Doctor
- Appointment
- Prescription

This satisfies Normalization and avoids redundancy.

**SQLITE PROGRAM:**

```
import sqlite3
conn = sqlite3.connect("hospital.db")
cur = conn.cursor()
# Create Tables
cur.execute("""
CREATE TABLE IF NOT EXISTS Patient(
PatientID INTEGER PRIMARY KEY,
Name TEXT,
Age INTEGER)
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS Doctor(
DoctorID INTEGER PRIMARY KEY,
DoctorName TEXT,
Specialization TEXT)
""")
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Appointment(
AppointmentID INTEGER PRIMARY KEY,
PatientID INTEGER,
DoctorID INTEGER,
Date TEXT,
FOREIGN KEY(PatientID) REFERENCES Patient(PatientID),
FOREIGN KEY(DoctorID) REFERENCES Doctor(DoctorID))
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS Prescription(
PrescriptionID INTEGER PRIMARY KEY,
AppointmentID INTEGER,
Medicine TEXT,
FOREIGN KEY(AppointmentID) REFERENCES Appointment(AppointmentID))
""")
# CREATE
cur.execute("INSERT OR IGNORE INTO Patient VALUES (1,'Rahul',25)")
cur.execute("INSERT OR IGNORE INTO Doctor VALUES (101,'Dr. Kumar','Cardiology')")
cur.execute("INSERT OR IGNORE INTO Appointment VALUES (1001,1,101,'2026-08-06')")
cur.execute("INSERT OR IGNORE INTO Prescription VALUES (5001,1001,'Paracetamol')")
conn.commit()
print("Patient Table")
for row in cur.execute("SELECT * FROM Patient"):
    print(row)
print("\nDoctor Table")
for row in cur.execute("SELECT * FROM Doctor"):
    print(row)
print("\nAppointment Table")
for row in cur.execute("SELECT * FROM Appointment"):
    print(row)
print("\nPrescription Table")
for row in cur.execute("SELECT * FROM Prescription"):
    print(row)
# UPDATE
cur.execute("UPDATE Patient SET Age=26 WHERE PatientID=1")
conn.commit()
print("\nAfter Update")
for row in cur.execute("SELECT * FROM Patient"):
    print(row)
# DELETE
cur.execute("DELETE FROM Prescription WHERE PrescriptionID=5001")
conn.commit()
print("\nAfter Delete Prescription")
for row in cur.execute("SELECT * FROM Prescription"):
    print(row)
conn.close()
```

**OUTPUT:**

```
Patient Table
(1, 'Rahul', 25)

Doctor Table
(101, 'Dr. Kumar', 'Cardiology')

Appointment Table
(1001, 1, 101, '2026-08-06')
```

```
Prescription Table
(5001, 1001, 'Paracetamol')

After Update
(1, 'Rahul', 26)

After Delete Prescription
```

**Q1. What problems might occur if prescription details are stored directly in the Appointment table?**
**Answer:**

- Data redundancy if multiple medicines are prescribed.
- Difficult to store more than one medicine in a single appointment.
- Update anomalies when medicines need modification.
- Increased null values if some appointments have no prescriptions.
- Violates database normalization principles.

**Q2. How would the schema change if a prescription contains multiple medicines?**
**Answer:**
Create a separate Medicine table (or Prescription_Medicine table) to represent the one-to-many relationship.
Example:
Prescription
---
PrescriptionID (PK)
AppointmentID (FK)

Medicine
---
MedicineID (PK)
PrescriptionID (FK)
MedicineName
Dosage
Frequency
This allows one prescription to include multiple medicines while maintaining normalization and avoiding redundancy.

**2. Food Delivery Application**

Tables:

- Customer(CustomerID, Name)
- Restaurant(RestaurantID, Name)
- Order(OrderID, CustomerID, RestaurantID)
- DeliveryAgent(AgentID, Name)
- Delivery(OrderID, AgentID)

Orders are delivered by delivery agents.

**Viva Questions**

Q1. Why is Delivery maintained as a separate table?

Q2. How would the schema change if multiple delivery agents could handle a single order?

**1. Identification of Entities and Attributes**

**Customer**

- **CustomerID** (Primary Key)
- Name

**Restaurant**

- **RestaurantID** (Primary Key)
- Name

**Orders**

- **OrderID** (Primary Key)
- CustomerID (Foreign Key)
- RestaurantID (Foreign Key)

**DeliveryAgent**

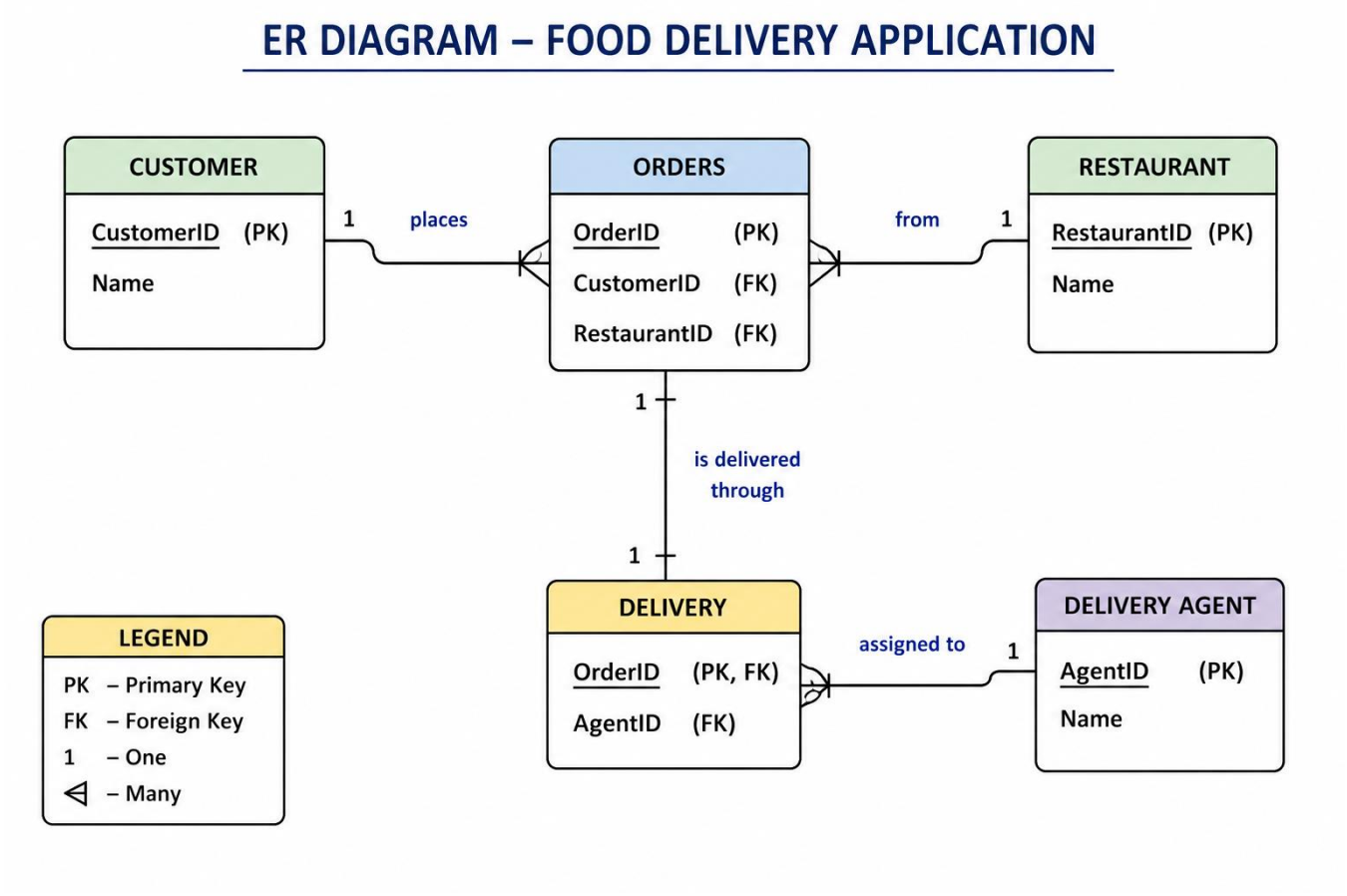
- **AgentID** (Primary Key)
- Name

**Delivery**

- OrderID (Foreign Key)
- AgentID (Foreign Key)

**2. Understanding of Relationships**

- Customer (1) → (M) Orders
- Restaurant (1) → (M) Orders
- DeliveryAgent (1) → (M) Delivery
- Orders (1) → (1) Delivery




**3. Keys and Constraints**

**Primary Keys**

- CustomerID
- RestaurantID
- OrderID
- AgentID

**Foreign Keys**

- Orders.CustomerID → Customer.CustomerID
- Orders.RestaurantID → Restaurant.RestaurantID
- Delivery.OrderID → Orders.OrderID
- Delivery.AgentID → DeliveryAgent.AgentID

**Constraints**

- Primary Keys must be unique.
- Foreign Keys maintain referential integrity.
- CustomerID, RestaurantID, OrderID, and AgentID cannot be NULL.

---

**4. Detection of Design Issues**

**Issue 1**

If Delivery Agent details are stored inside the Orders table:

- Data redundancy occurs.
- The same agent information is repeated for every order.

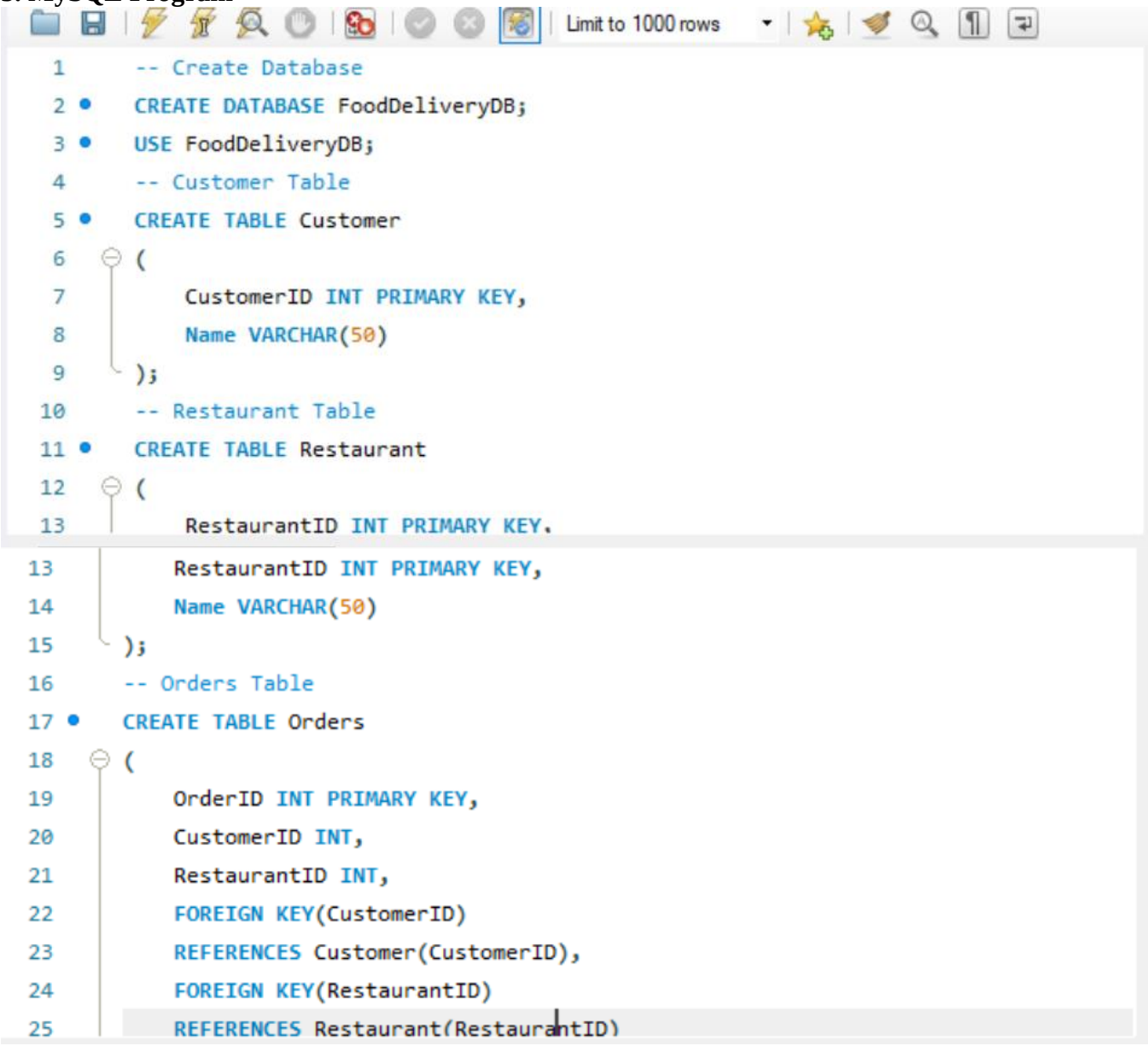
**Issue 2**

If an agent's name changes:

- Every order record must be updated.
- This causes update anomalies.

**Solution**
Create a separate **Delivery** table that links **Orders** and **DeliveryAgent**.

**5. MySQL Program**



```
25          REFERENCES Restaurant(RestaurantID)
26      );
27      -- Delivery Agent Table
28      CREATE TABLE DeliveryAgent
29      (
30          AgentID INT PRIMARY KEY,
31          Name VARCHAR(50)
32      );
33      -- Delivery Table
34      CREATE TABLE Delivery
35      (
36          OrderID INT,
37          AgentID INT,
```

```
37          AgentID INT,
38          PRIMARY KEY(OrderID),
39          FOREIGN KEY(OrderID)
40          REFERENCES Orders(OrderID),
41          FOREIGN KEY(AgentID)
42          REFERENCES DeliveryAgent(AgentID)
43      );
44      INSERT INTO Customer
45      VALUES
46      (1,'Rahul'),
47      (2,'Priya');
48      INSERT INTO Restaurant
49      VALUES
```

```
49      VALUES
50      (101,'Dominos'),
51      (102,'KFC');
52      INSERT INTO Orders
53      VALUES
54      (1001,1,101),
55      (1002,2,102);
56      INSERT INTO DeliveryAgent
57      VALUES
58      (501,'Arun'),
59      (502,'Vijay');
60      INSERT INTO Delivery
61      VALUES
```

```
61      VALUES
62      (1001,501),
63      (1002,502);
64  •   SELECT * FROM Customer;
65  •   SELECT * FROM Restaurant;
66  •   SELECT * FROM Orders;
67  •   SELECT * FROM DeliveryAgent;
68  •   SELECT * FROM Delivery;
69  •   UPDATE DeliveryAgent
70      SET Name='Ramesh'
71      WHERE AgentID=501;
72  •   SELECT * FROM DeliveryAgent;
73  •   DELETE FROM Delivery
```

```
72  •   SELECT * FROM DeliveryAgent;
73  •   DELETE FROM Delivery
74      WHERE OrderID=1002;
75  •   SELECT * FROM Delivery;
```

**READ:**

| CustomerID | Name |
|---|---|
| 1 | Rahul |
| 2 | Priya |
| NULL | NULL |

| RestaurantID | Name |
|---|---|
| 101 | Dominos |
| 102 | KFC |
| NULL | NULL |

| OrderID | CustomerID | RestaurantID |
|---|---|---|
| 1001 | 1 | 101 |
| 1002 | 2 | 102 |
| NULL | NULL | NULL |

| AgentID | Name |
|---|---|
| 501 | Ramesh |
| 502 | Vijay |
| NULL | NULL |

| OrderID | AgentID |
|---|---|
| 1001 | 501 |
| 1002 | 502 |
| NULL | NULL |

**UPDATE:**

| | AgentID | Name |
|---|---|---|
| ▶ | 501 | Ramesh |
| | 502 | Vijay |
| * | NULL | NULL |

**DELETE**

| | OrderID | AgentID |
|---|---|---|
| ▶ | 1001 | 501 |
| * | NULL | NULL |

**Viva Questions**

**Q1. Why is Delivery maintained as a separate table?**

**Answer:**

- It avoids storing delivery agent details repeatedly in the Orders table.
- It reduces data redundancy.
- It prevents update and deletion anomalies.
- It maintains database normalization.
- It makes it easier to manage delivery assignments independently.

**Q2. How would the schema change if multiple delivery agents could handle a single order?**

**Answer:**

The relationship changes from **One Order → One Delivery Agent** to **Many-to-Many (Orders ↔ DeliveryAgent)**.

The **Delivery** table becomes a junction table with a **composite primary key**:

```
CREATE TABLE Delivery
(
   OrderID INT,
   AgentID INT,

   PRIMARY KEY (OrderID, AgentID),

   FOREIGN KEY (OrderID)
      REFERENCES Orders(OrderID),

   FOREIGN KEY (AgentID)
      REFERENCES DeliveryAgent(AgentID)
);
```

**Example data:**

| OrderID | AgentID |
|---|---|
| 1001 | 501 |
| 1001 | 502 |
| 1002 | 503 |

This design allows multiple delivery agents to work on the same order while maintaining proper normalization.